Se former autrement

AI Agent Evolution Lab

Plateforme de création, d'orchestration, d'évaluation, d'optimisation et de supervision d'agents IA

Rapport technique

Mini-projet — Architecture microservices

Réalisé par :

Anaghim Deraoui
Souleima Belguith
Ouday Chebbi
Roua Ouerfelli
Maram Ben Rhim
Youssef Souilem

Année universitaire 2026–2027

Novembre 2026

Résumé

Les agents d'intelligence artificielle savent aujourd'hui accomplir des tâches complexes, mais ils restent le plus souvent isolés, difficiles à mesurer et longs à améliorer. Ce rapport présente la conception de **AI Agent Evolution Lab**, une plateforme web qui permet de *créer*, *orchestrer*, *évaluer*, *optimiser* et *superviser* des agents IA au sein d'un même système collaboratif.

La conception proposée repose sur une architecture **microservices** composée de six services métier (gestion des agents, outils MCP et sécurité, gestion des missions, benchmark et sécurité, optimisation, détection d'anomalies) et d'un service commun de traçabilité. Les échanges synchrones utilisent REST et OpenFeign, tandis que la journalisation s'appuie sur des événements asynchrones via RabbitMQ. Chaque service possède sa propre base de données. L'infrastructure comprend une API Gateway, Keycloak, Eureka, un Config Server et Docker.

Un scénario illustratif sur 20 fichiers de ventes présente une progression hypothétique de 60 % à 85 % de rapports conformes. Ces chiffres sont fictifs et ne constituent pas des résultats mesurés.

Mots-clés : agents IA, microservices, orchestration multi-agents, LangGraph, MCP, Optuna, détection d'anomalies, RabbitMQ, Spring Boot.

Conformité aux consignes de rendu

Le présent document technique accompagne la présentation du projet et couvre la description du sujet, les acteurs et fonctionnalités, le découpage en microservices, les données gérées, les interactions, le diagramme d'architecture globale et les choix complémentaires de conception.

Dépôt Git public : *lien à compléter par l'équipe avant le dépôt Blackboard.* L'enseignant devra être ajouté comme collaborateur conformément aux consignes.

Pièces à déposer sur Blackboard : lien Git public, présent rapport technique au format PDF et présentation de l'équipe. **Un seul membre** effectue le dépôt pour l'ensemble de l'équipe.

Portée du document : architecture et contrats d'API proposés ; les exemples de charges utiles et de résultats sont illustratifs tant que l'intégration et les tests réels ne sont pas terminés.

Table des matières

Résumé **1**

Conformité aux consignes de rendu **2**

1 Introduction et problématique **8**
1.1 Contexte . . . 8
1.2 Problématique . . . 8
1.3 Objectifs . . . 8
1.4 Organisation du rapport . . . 8

2 Présentation du projet **9**
2.1 Vue d'ensemble . . . 9
2.2 Scénario d'usage : analyse de ventes . . . 9
2.3 Illustration de l'optimisation (simulation) . . . 9

3 Analyse fonctionnelle **11**
3.1 Acteurs . . . 11
3.2 Cas d'utilisation . . . 11

4 Découpage en microservices **13**

5 Données et communications **14**
5.1 Principe : une base par service . . . 14
5.2 Communications synchrones . . . 14
5.3 Frontières des données et responsabilités . . . 14
5.4 Communication asynchrone . . . 15

6 Architecture technique **16**
6.1 Architecture cible . . . 16
6.2 Composants d'infrastructure . . . 16

7 Conception détaillée des microservices **17**
7.1 M1 — Agent Management . . . 17
7.1.1 Responsabilité et modèle . . . 17
7.1.2 API REST . . . 17
7.1.3 Interactions . . . 17
7.2 M2 — MCP & Security . . . 18
7.2.1 Responsabilité et modèle . . . 18
7.2.2 API REST . . . 18
7.2.3 Communications . . . 18
7.3 M3 — Mission Management . . . 19
7.3.1 Responsabilité et modèle . . . 19
7.3.2 API REST et codes de réponse . . . 20

7.3.3 Interactions . . . 20
7.3.4 Pannes et résilience . . . 20
7.4 M4 — Benchmark & Safety . . . 21
7.4.1 Responsabilité et modèle . . . 21
7.4.2 API REST . . . 22
7.4.3 Communications et pannes . . . 22
7.5 M5 — Optimization (Optuna) . . . 23
7.5.1 Responsabilité et modèle . . . 23
7.5.2 API REST et événements . . . 24
7.5.3 Choix des modes de communication . . . 24
7.5.4 Gestion des pannes . . . 25
7.6 M6 — Monitoring & Anomaly Detection . . . 26
7.6.1 Responsabilité . . . 26
7.6.2 Modèle de données . . . 27
7.6.3 Diagnostic et décision de gravité . . . 27
7.6.4 API REST et événements . . . 27
7.6.5 Gestion des pannes . . . 28
7.7 M7 — Event Logging & Analytics . . . 29
7.7.1 Responsabilité et modèle . . . 29
7.7.2 Architecture interne et persistance . . . 30
7.7.3 Exemple de déroulement de bout en bout . . . 30
7.7.4 Événements consommés . . . 30
7.7.5 API REST (lecture seule) . . . 30
7.7.6 Gestion des pannes . . . 31

8 Sécurité 32

9 Résilience et gestion des pannes 33

10 Risques et solutions 34

11 Technologies du projet 35

12 Planning du mini-projet 36

13 Conclusion et perspectives 37

A Exemples de messages échangés 38
A.1 Création d'une mission (Frontend/Gateway → M3) . . . 38
A.2 Exécution d'un agent (M3 → M1) . . . 38
A.3 Contrôle technique (M3 → M6) . . . 38
A.4 Événement de fin de mission (M3 → RabbitMQ → M7) . . . 38
A.5 Lancement d'un run de benchmark (M5 → M4) . . . 38
A.6 Lancement d'une étude d'optimisation (Gateway → M5) . . . 39
A.7 Application d'une configuration (M5 → M1) . . . 39
A.8 Événements de M5 vers M7 . . . 39
A.9 Diagnostic M6 → M3 . . . 39

A.10 Consultation de M7 39

Table des figures

2.1 Scénario illustratif de collaboration de trois agents (diapositive 10) 9

3.1 Diagramme de cas d'utilisation original (diapositive 16) 12

6.1 Architecture globale originale : services, bases et infrastructure (diapositive 20) 16

7.1 Interactions de M1 avec le front-end et les autres microservices (diapositive 23) 18
7.2 Diagramme de classes de M2 issu de la présentation (diapositive 24) 19
7.3 Communications REST, MCP et RabbitMQ pour M2 (diapositive 26) 19
7.4 Diagramme de classes de M3 : missions, étapes, participants et exécutions (diapositive 27) 21
7.5 Interactions de M3 : M1, M6 et événements RabbitMQ vers M7 (diapositive 28) . . . 21
7.6 Modèle de classes de M4 : benchmarks, jeux de test et évaluations (diapositive 31) . . 23
7.7 Flux de benchmarking entre M4, M1 et M5 (diapositive 32) 23
7.8 Diagramme de classes de M5 : études, essais et configurations (diapositive 35) 26
7.9 Échanges de M5 avec M4, M1 et M7 (diapositive 34) 26
7.10 Modèle de classes de M6 : métriques, diagnostics et alertes (diapositive 40) 29
7.11 Interactions de surveillance de M6 avec M3, M1 et M7 (diapositive 39) 29
7.12 Chaîne de traçabilité : événements métier, RabbitMQ, M7 et MongoDB (diapositive 43) 31

Liste des tableaux

1.1 Objectifs de la plateforme 8

2.1 Illustration fictive du taux de conformité avant et après optimisation 10

3.1 Cas d'utilisation par acteur 11

4.1 Découpage en microservices 13

5.1 Base de données dédiée à chaque microservice 14
5.2 Échanges synchrones entre microservices 14

6.1 Rôle des composants d'infrastructure 16

7.1 Entités principales de M1 17
7.2 API REST de M1 17
7.3 Entités principales de M2 18
7.4 API REST de M2 18
7.5 Entités principales de M3 19
7.6 API REST de M3 (contrats proposés) 20
7.7 Entités principales de M4 22
7.8 API REST de M4 (préfixe /api) 22
7.9 Entités principales de M5 24
7.10 API REST de M5 24
7.11 Gestion des pannes de M5 25
7.12 Entités principales de M6 27
7.13 Décision de gravité et action de M3 27
7.14 API REST de M6 27
7.15 Gestion des pannes de M6 28
7.16 Événements publiés dans RabbitMQ 30
7.17 API REST de M7 30
7.18 Gestion des pannes de M7 31

9.1 Synthèse de la résilience par dépendance 33

10.1 Risques prioritaires et mesures 34

11.1 Stack technique de AI Agent Evolution Lab 35

12.1 Feuille de route (séances 4 à 10) 36

1 Introduction et problématique

1.1 Contexte

Les agents IA, capables de raisonner, d'utiliser des outils et d'enchaîner des actions, se multiplient dans les entreprises. Pourtant, dans la pratique, plusieurs agents coexistent sans coordination claire : chacun est configuré de son côté, sa qualité n'est pas mesurée de façon systématique, et les anomalies sont découvertes une fois la réponse publiée.

1.2 Problématique

> *Comment transformer ces agents indépendants en un système collaboratif, performant et évolutif, capable d'accomplir des missions complexes, d'améliorer continuellement ses résultats et de détecter les anomalies avant la publication des réponses ?*

Trois limites principales motivent le projet :

- **Isolement** : absence de collaboration structurée entre agents ;
- **Manque de mesure** : pas de score officiel ni de comparaison entre versions d'agents ;
- **Amélioration lente** : le réglage des configurations (température, nombre de tokens, style, etc.) reste manuel.

1.3 Objectifs

Le projet vise quatre objectifs, présentés dans le tableau 1.1.

Table 1.1 – Objectifs de la plateforme

N°	Axe	Description
01	Création	Configurer et gérer des agents IA
02	Orchestration	Faire collaborer plusieurs agents sur une même mission
03	Amélioration	Évaluer et optimiser leurs performances
04	Fiabilité	Détecter les anomalies avant la réponse finale

1.4 Organisation du rapport

Le chapitre 2 présente le projet et un scénario d'usage. Le chapitre 3 décrit l'analyse fonctionnelle. Les chapitres 4 à 6 détaillent le découpage en microservices, les données, les communications et l'architecture technique. Le chapitre 7 présente la conception détaillée de chaque microservice. Enfin, les chapitres suivants traitent de la sécurité, de la résilience, des risques, des technologies et du planning.

2 Présentation du projet

2.1 Vue d'ensemble

AI Agent Evolution Lab est une plateforme web intelligente pour créer, orchestrer et améliorer les agents IA. Une *mission* confiée à la plateforme mobilise plusieurs agents ; elle est configurée, exécutée, évaluée, optimisée et surveillée, tandis que tous les événements sont conservés dans un historique centralisé.

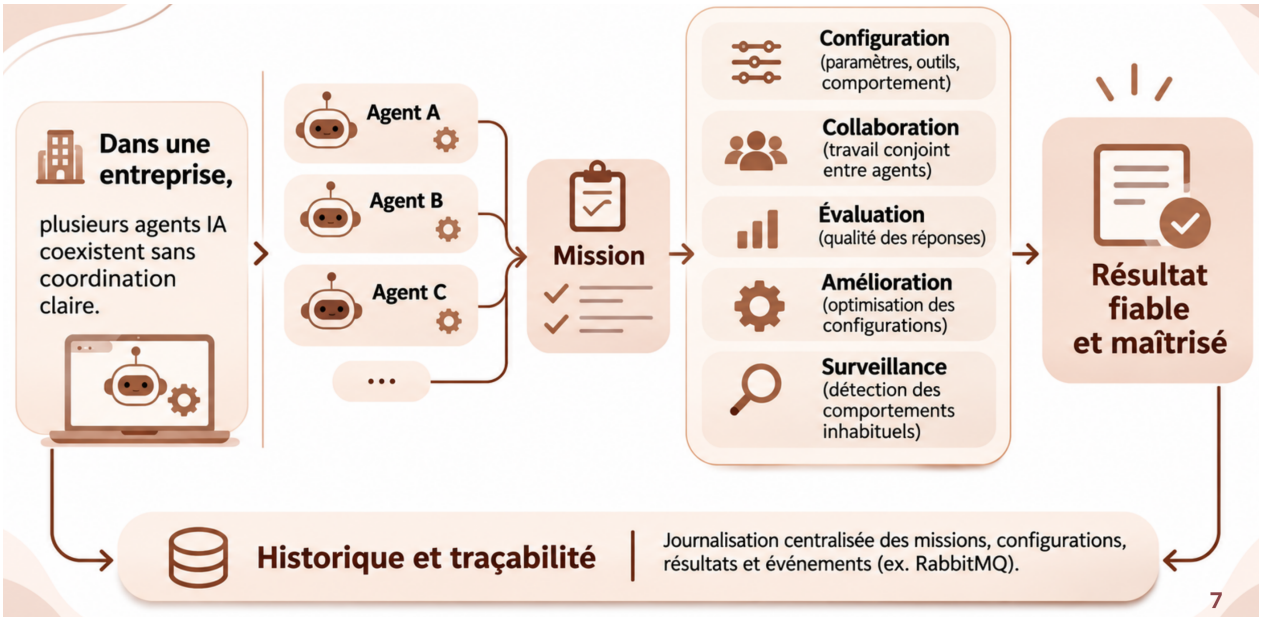


Figure 2.1 – Scénario illustratif de collaboration de trois agents (diapositive 10)

2.2 Scénario d'usage : analyse de ventes

Pour illustrer la valeur de la plateforme, considérons la demande suivante :

> *« Analysez mes ventes, repérez les anomalies et livrez un rapport illustré et vérifié. »*

Trois agents collaborent successivement :

1. **Agent Préparateur** : nettoie les données et contrôle leur qualité ;
2. **Agent Analyste** : analyse les ventes et crée des graphiques ;
3. **Agent Rapporteur** : rédige un rapport et vérifie sa cohérence.

Les microservices interviennent ainsi : **M1** fournit les agents, **M2** autorise leurs outils, **M3** organise leurs étapes et **M6** surveille l'exécution. Ensuite, **M4** mesure leur qualité, **M5** cherche de meilleurs réglages et **M7** conserve les événements.

2.3 Illustration de l'optimisation (simulation)

La présentation propose un **exemple fictif** sur 20 fichiers de ventes avec résultats attendus. Il sert à expliquer comment M4 mesure la qualité et comment M5 pourrait améliorer les configurations. **Aucune mesure réelle n'est revendiquée à ce stade.**

Table 2.1 – Illustration fictive du taux de conformité avant et après optimisation

Situation	**Rapports conformes**	**Taux**
Avant optimisation	12 / 20	60 %
Après optimisation	17 / 20	85 %
Gain	+5 rapports	**+25 points**

M4 évalue les rapports ; **M5** teste plusieurs configurations et propose la meilleure. *Les chiffres de ce tableau sont donnés uniquement à titre pédagogique ; les gains réels restent à mesurer.*

3 Analyse fonctionnelle

3.1 Acteurs

Deux acteurs interagissent avec la plateforme :

— **Utilisateur** : crée et utilise les agents et les missions, consulte les résultats ;
— **Administrateur** : gère les utilisateurs et supervise la plateforme.

3.2 Cas d'utilisation

Table 3.1 – Cas d'utilisation par acteur

Acteur	Cas d'utilisation
Utilisateur	Gérer ses agents IA
Utilisateur	Exécuter un agent IA
Utilisateur	Gérer ses missions
Utilisateur	Exécuter une mission multi-agents
Utilisateur	Associer des outils autorisés aux agents
Utilisateur	Évaluer les agents
Utilisateur	Consulter les résultats et les scores
Utilisateur	Optimiser les configurations des agents
Utilisateur	Consulter les alertes d'anomalies
Administrateur	Gérer les utilisateurs et permissions
Administrateur	Superviser la plateforme et les événements

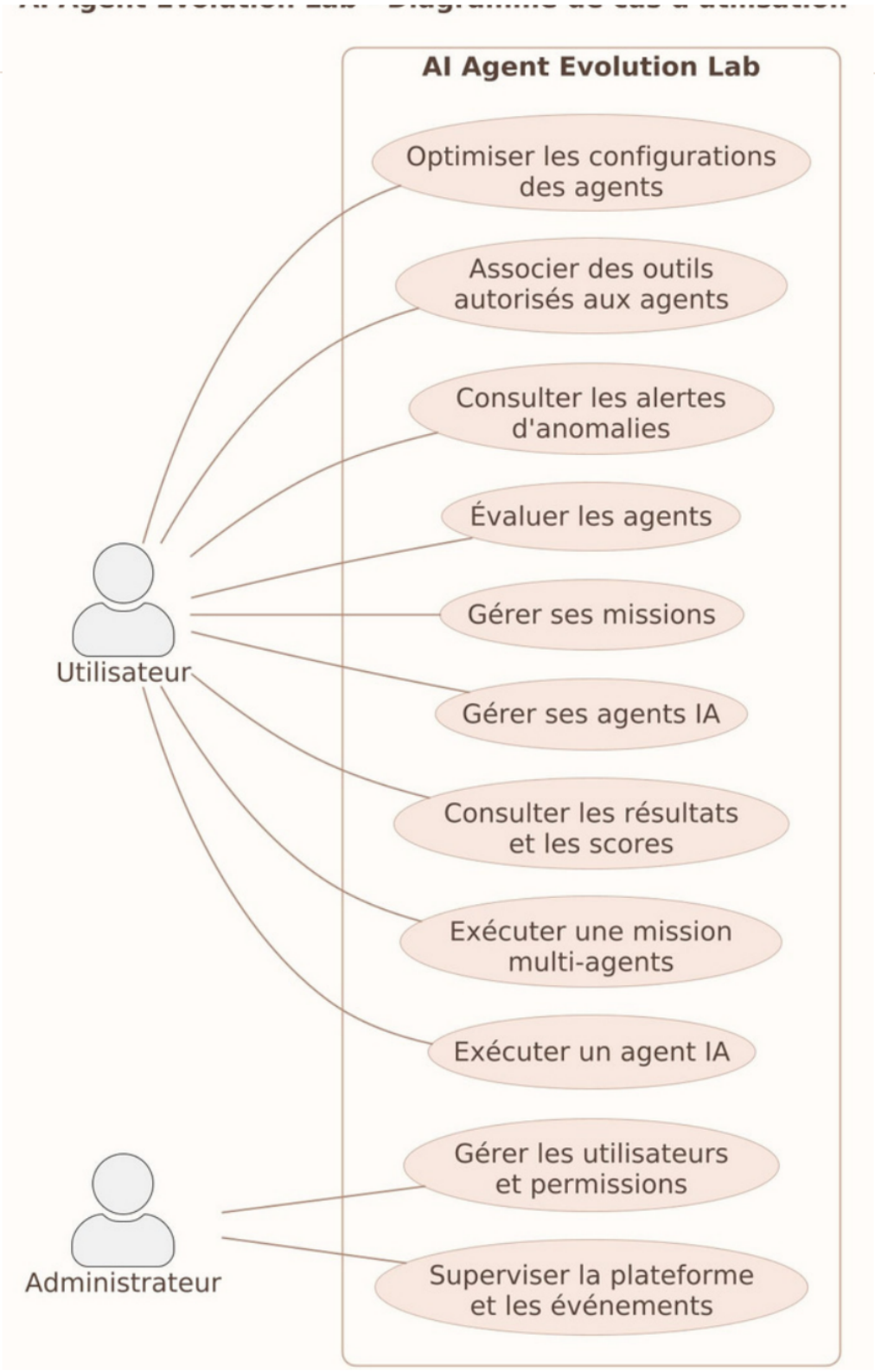


Figure 3.1 – Diagramme de cas d'utilisation original (diapositive 16)

4 Découpage en microservices

Le système est découpé en **six responsabilités métier distinctes** et **un microservice commun** de traçabilité. Ce découpage isole les responsabilités, permet à chaque membre de l'équipe de travailler sur un service précis et limite l'impact d'une panne.

Table 4.1 – Découpage en microservices

ID	Microservice	Responsabilité
M1	Agent Management	Créer, configurer et exécuter les agents IA
M2	MCP & Security	Gérer les outils et contrôler les permissions
M3	Mission Management	Orchestrer les missions avec LangGraph
M4	Benchmark & Safety	Tester les agents et mesurer leur qualité
M5	Optimization (Optuna)	Rechercher les meilleures configurations
M6	Monitoring & Anomaly Detection	Détecter les anomalies avant la réponse
M7	Event Logging & Analytics	Service commun : centralisation des événements via RabbitMQ

5 Données et communications

5.1 Principe : une base par service

Chaque microservice reste propriétaire de ses données et expose ses API aux autres services. **Aucune base de données n'est partagée** et aucune jointure SQL n'existe entre services : les références entre services (par exemple l'identifiant d'un agent dans **M3**) sont de simples identifiants résolus par appel d'API.

Table 5.1 – Base de données dédiée à chaque microservice

ID	Microservice	Base	Type
M1	Agent Management	DB1 — H2	Relationnelle
M2	MCP & Security	DB2 — MySQL	Relationnelle
M3	Mission Management	DB3 — MySQL	Relationnelle
M4	Benchmark & Safety	DB4 — MySQL	Relationnelle
M5	Optimization (Optuna)	DB5 — PostgreSQL	Relationnelle + JSONB
M6	Monitoring & Anomaly Detection	DB6 — H2	Relationnelle
M7	Event Logging & Analytics	DB7 — MongoDB	Documents (JSON)

5.2 Communications synchrones

Les échanges synchrones utilisent **REST** avec **OpenFeign**. Ils sont employés lorsque l'appelant a besoin de la réponse pour poursuivre.

Table 5.2 – Échanges synchrones entre microservices

Services	Sens	Objet
M1 ↔ M2	bidirectionnel	Utiliser les outils MCP autorisés
M3 ↔ M1	bidirectionnel	Récupérer et exécuter les agents IA
M3 ↔ M6	bidirectionnel	Détecter les anomalies avant la réponse finale
M6 ↔ M1	bidirectionnel	Consulter des informations complémentaires (si nécessaire)
M4 ↔ M1	bidirectionnel	Exécuter les agents sur des jeux de tests
M5 ↔ M4	bidirectionnel	Demander les évaluations et récupérer les scores
M5 → M1	unidirectionnel	Consulter et proposer des configurations optimisées

5.3 Frontières des données et responsabilités

Les identifiants échangés entre microservices ne constituent pas des clés étrangères physiques entre les bases. Par exemple, M3 conserve un `agentId` désignant un agent de M1 : il vérifie son existence par l'API de M1 au moment de la création ou de l'exécution de la mission. Cela évite de coupler les schémas SQL, mais nécessite la gestion des suppressions, des interruptions réseau et de l'éventuelle obsolescence d'un identifiant.

Pour une opération qui implique plusieurs services, aucune transaction SQL globale n'est supposée : chaque microservice valide ses propres données. Une erreur d'exécution est matérialisée dans l'état de la mission et signalée par un événement, tandis que M7 maintient un historique consultable. Les messages RabbitMQ transportent des identifiants et des métadonnées techniques ; ils ne remplacent pas les API métiers.

5.4 Communication asynchrone

Les services **M1** à **M6** publient leurs événements dans **RabbitMQ** (exchange `evolution.events`), consommés par **M7** pour la journalisation des événements et des anomalies. Ce mode vise à découpler la traçabilité de l'exécution métier. Une gestion des reprises, de la persistance et des erreurs reste nécessaire pour assurer une livraison fiable.

6 Architecture technique

6.1 Architecture cible

L'ensemble est déployé sur un hôte Docker. Le front-end React communique uniquement avec l'**API Gateway** (Spring Cloud Gateway), qui délègue l'authentification et l'autorisation à **Keycloak** et route les requêtes vers les microservices enregistrés dans **Eureka**. Un **Config Server** centralise la configuration.

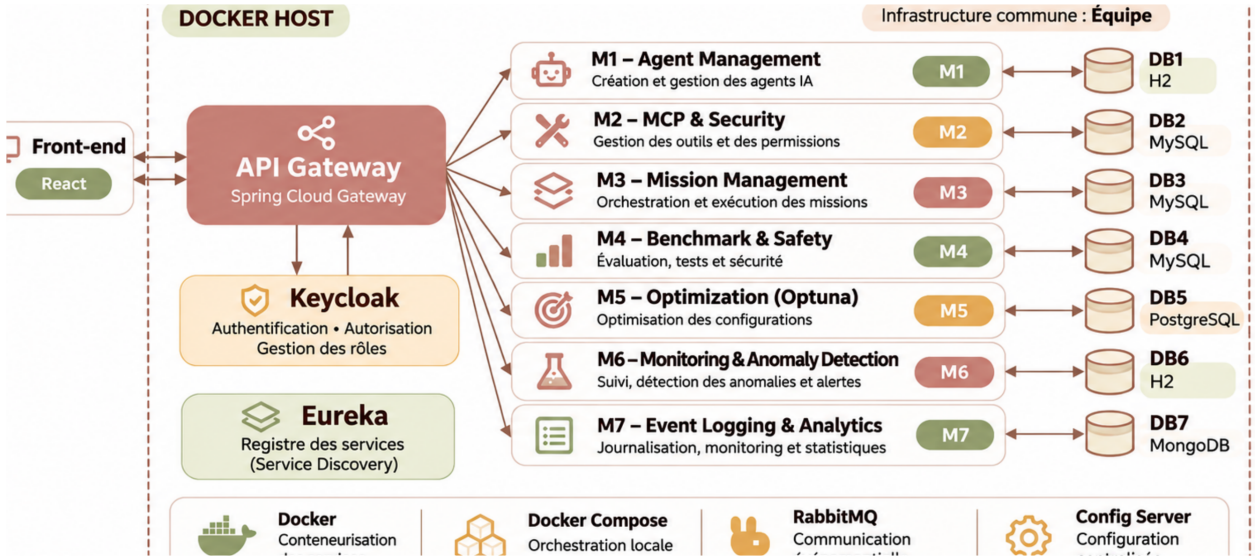


Figure 6.1 – Architecture globale originale : services, bases et infrastructure (diapositive 20)

6.2 Composants d'infrastructure

Table 6.1 – Rôle des composants d'infrastructure

Composant	Rôle
Spring Cloud Gateway	Point d'entrée unique, routage vers les microservices
Keycloak	Authentification, autorisation, gestion des rôles (JWT)
Eureka	Registre des services (Service Discovery)
Config Server	Configuration centralisée
RabbitMQ	Communication événementielle asynchrone
Docker / Docker Compose	Conteneurisation des services et orchestration locale

7 Conception détaillée des microservices

7.1 M1 — Agent Management

7.1.1 Responsabilité et modèle

M1 gère le catalogue, les versions et l'exécution des agents. Un agent possède un nom, une description, un propriétaire et un statut (`DRAFT`, `ACTIVE` ou `ARCHIVED`). Chaque modification crée une *version* (modèle, prompt système, historique des changements). Chaque version est associée à une *configuration* : température, nombre maximal de tours, délai d'expiration et outils autorisés.

Les outils sont référencés par des identifiants provenant de **M2**, sans jointure SQL. La base est H2, propre à **M1**.

Table 7.1 – Entités principales de M1

Entité	Attributs
`Agent`	id, name, description, ownerId, status, createdAt, updatedAt
`AgentVersion`	id, versionNumber, model, systemPrompt, changeLog, active, createdAt
`AgentConfiguration`	id, temperature, maxTurns, timeoutSeconds, allowedToolIds
`AgentStatus` (enum)	DRAFT, ACTIVE, ARCHIVED

Relations : un `Agent` possède 0..∗ `AgentVersion` (composition) ; chaque `AgentVersion` est liée à exactement une `AgentConfiguration`.

7.1.2 API REST

Table 7.2 – API REST de M1

Méthode	URI	Action
POST	`/api/agents`	Créer un agent
GET	`/api/agents`	Lister les agents
GET	`/api/agents/{id}`	Détail d'un agent
PUT	`/api/agents/{id}`	Modifier un agent
DELETE	`/api/agents/{id}`	Supprimer un agent
POST	`/api/agents/{id}/versions`	Créer une version
GET	`/api/agents/{id}/versions`	Lister les versions
GET	`/api/agents/{id}/versions/{vid}`	Détail d'une version
PUT	`/api/agents/{id}/versions/{vid}/activate`	Activer une version
PUT	`/api/agents/{id}/versions/{vid}/configuration`	Modifier la configuration
POST	`/api/agents/{id}/execute`	Exécuter un agent

7.1.3 Interactions

- **M2** : autorisation des outils (`POST /tools/authorize`) ;
- **M3** : catalogue (`GET /api/agents`) et exécution (`POST /api/agents/{id}/execute`) ;
- **M4** : exécution sur jeux de tests ;
- **M5** : lecture et création de versions et configurations ;
- **M6** : lecture d'une version (`GET /api/agents/{id}/versions/{vid}`) ;
- **M7** (asynchrone) : événements `AGENT_CREATED`, `AGENT_EXECUTED`, `AGENT_UPDATED`.

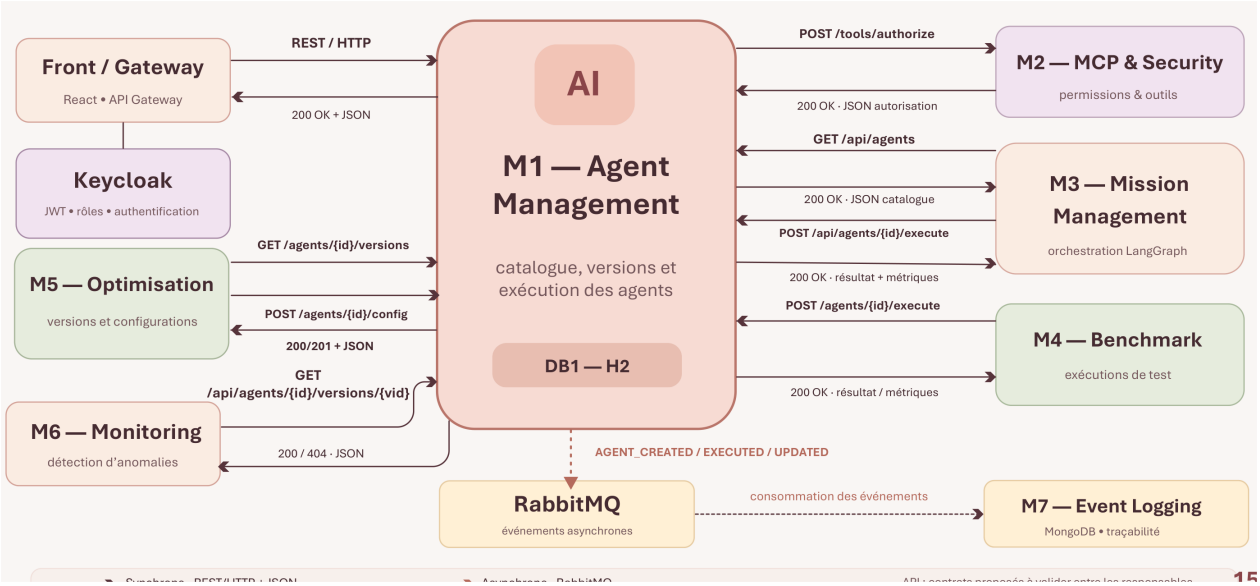


Figure 7.1 – Interactions de M1 avec le front-end et les autres microservices (diapositive 23)

7.2 M2 — MCP & Security

7.2.1 Responsabilité et modèle

M2 (Tool & Skill Management) gère les outils, intègre les serveurs MCP (Model Context Protocol) et contrôle leur utilisation par les agents IA. Sa base est MySQL.

Table 7.3 – Entités principales de M2

Entité	Attributs / rôle
`ServeurMCP`	id, nom, adresse, typeTransport, etat — enregistre et modifie un serveur
`OutilMCP`	id, nom, description, schemaEntree, actif — fourni par un serveur ($1 \rightarrow 0..*$)
`Autorisation`	id, identifiantAgent, autorisee, dateCreation — concerne un outil ; `verifier()`
`PolitiqueAcces`	id, nom, description, active — définit des autorisations ($1 \rightarrow 0..*$)

7.2.2 API REST

Table 7.4 – API REST de M2

Méthode	Endpoint	Fonction
POST	`/api/tool-connections`	Ajouter une connexion MCP
POST	`/api/tool-connections/{id}/discover`	Découvrir les outils
GET	`/api/tools`	Consulter le catalogue
POST	`/api/tools/{id}/permissions`	Gérer les permissions
POST	`/api/tools/{id}/execute`	Exécuter un outil

7.2.3 Communications

1. **M1** $\leftrightarrow$ **M2** (REST) : demande et utilisation des outils par les agents IA ;
2. **M2** $\leftrightarrow$ **M7** (RabbitMQ) : transmission des événements d'exécution ;
3. **M2** $\leftrightarrow$ serveur MCP (protocole MCP) : découverte et exécution des outils externes.

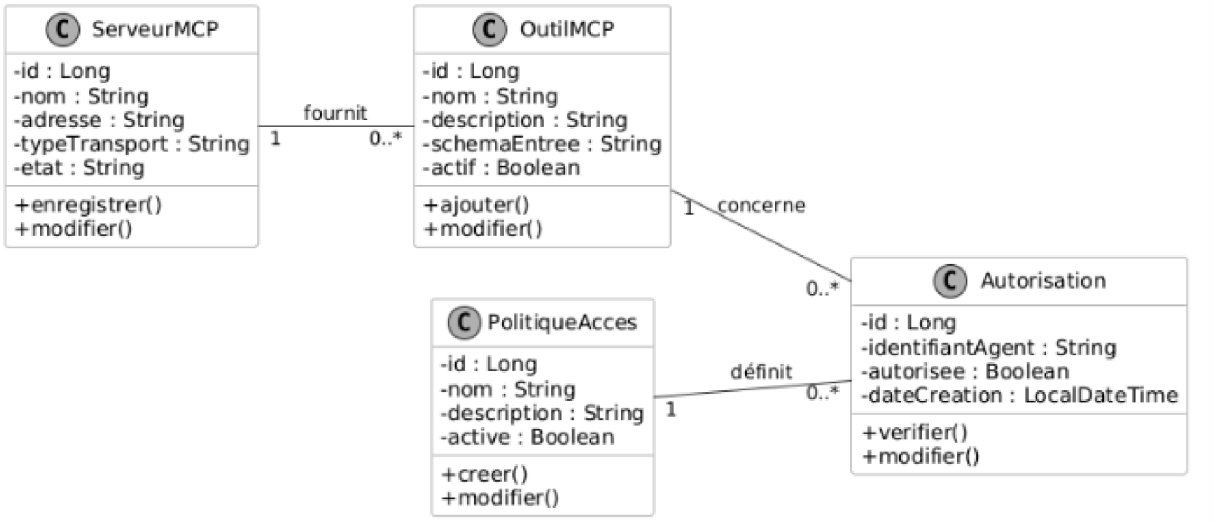


Figure 7.2 – Diagramme de classes de M2 issu de la présentation (diapositive 24)

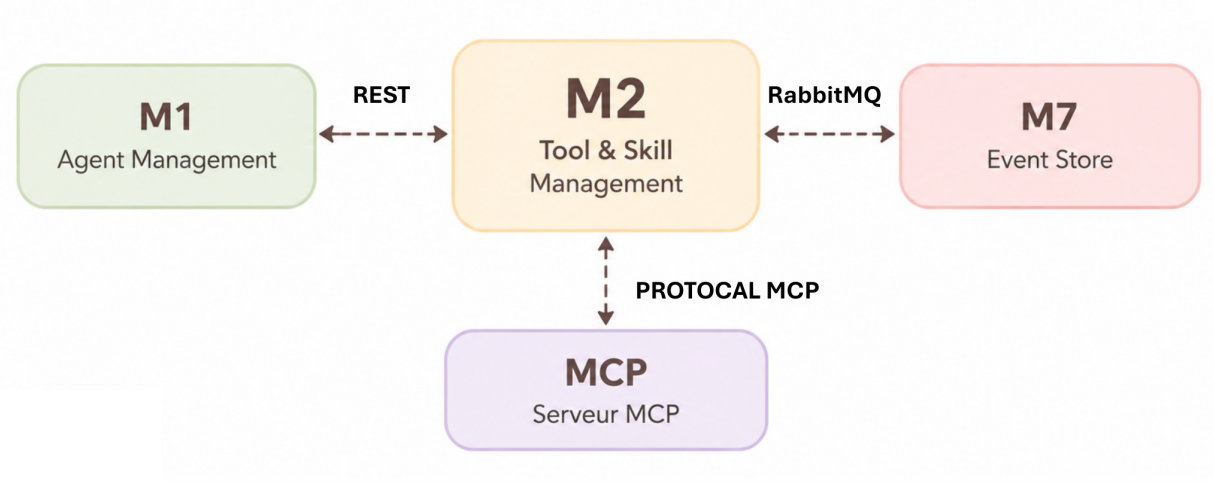


Figure 7.3 – Communications REST, MCP et RabbitMQ pour M2 (diapositive 26)

7.3 M3 — Mission Management

7.3.1 Responsabilité et modèle

M3 crée et orchestre les missions (via LangGraph) et suit leurs exécutions. Sa base est MySQL.

Table 7.5 – Entités principales de M3

Entité	Attributs
`Mission`	id, name, objective, selectionMode, maxAttempts, createdAt
`MissionStep`	id, stepOrder, name, agentVersionId
`MissionParticipant`	id, agentId, agentVersionId, role
`MissionExecution`	id, status, finalResult, totalTokensUsed, startedAt, finishedAt
`SelectionMode` (enum)	MANUAL, AUTOMATIC_AI
`ExecutionStatus` (enum)	PENDING, RUNNING, COMPLETED, FAILED

Une `Mission` contient 0..∗ étapes, possède 0..∗ participants et conserve l'historique de 0..∗ exécutions.

7.3.2 API REST et codes de réponse

Table 7.6 – API REST de M3 (contrats proposés)

HTTP	Endpoint	Rôle métier	Réponse
POST	`/api/missions`	Créer une mission	201
GET	`/api/missions`	Lister les missions	200
GET	`/api/missions/{id}`	Consulter une mission	200 / 404
PUT	`/api/missions/{id}`	Modifier une mission	200 / 400 / 404
DELETE	`/api/missions/{id}`	Supprimer une mission	204 / 404
POST	`/api/missions/{id}/execute`	Démarrer une exécution	202 / 409 / 503
GET	`/api/executions/{id}`	Suivre le statut et le résultat	200 / 404

7.3.3 Interactions

M3 appelle **M1** (catalogue et exécution des agents, retour : résultat, tokens, durée) et **M6** (diagnostic d'anomalie, retour : `severity`, `blocking`). Il publie `MISSION_CREATED`, `MISSION_COMPLETED` et `MISSION_FAILED` vers **M7** via RabbitMQ. Les outils MCP sont appelés par **M1**, pas directement par **M3**.

7.3.4 Pannes et résilience

— **M1 indisponible** : timeout et nouvelles tentatives bornées ; mission en attente ou `FAILED` ;
— **M6 indisponible** : aucun diagnostic inventé ; statut provisoire ou échec selon la politique ;
— **M7 indisponible** : RabbitMQ découple la journalisation ; l'échec de publication doit être géré.

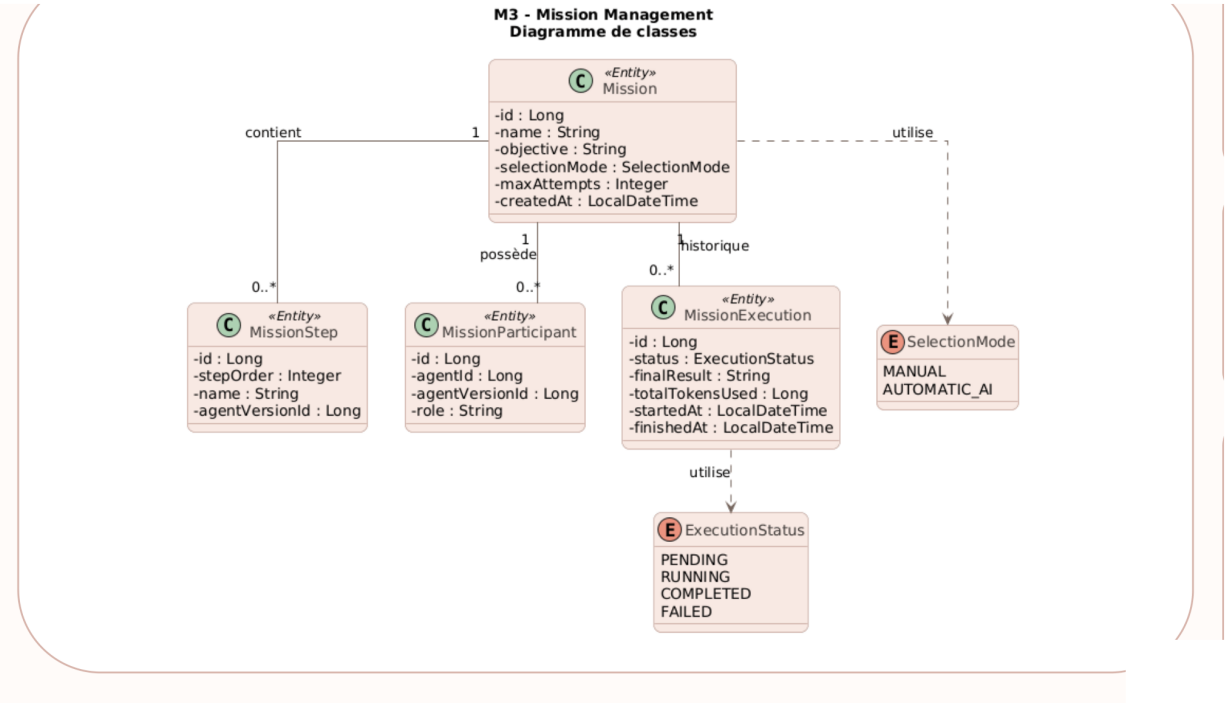


Figure 7.4 – Diagramme de classes de M3 : missions, étapes, participants et exécutions (diapositive 27)

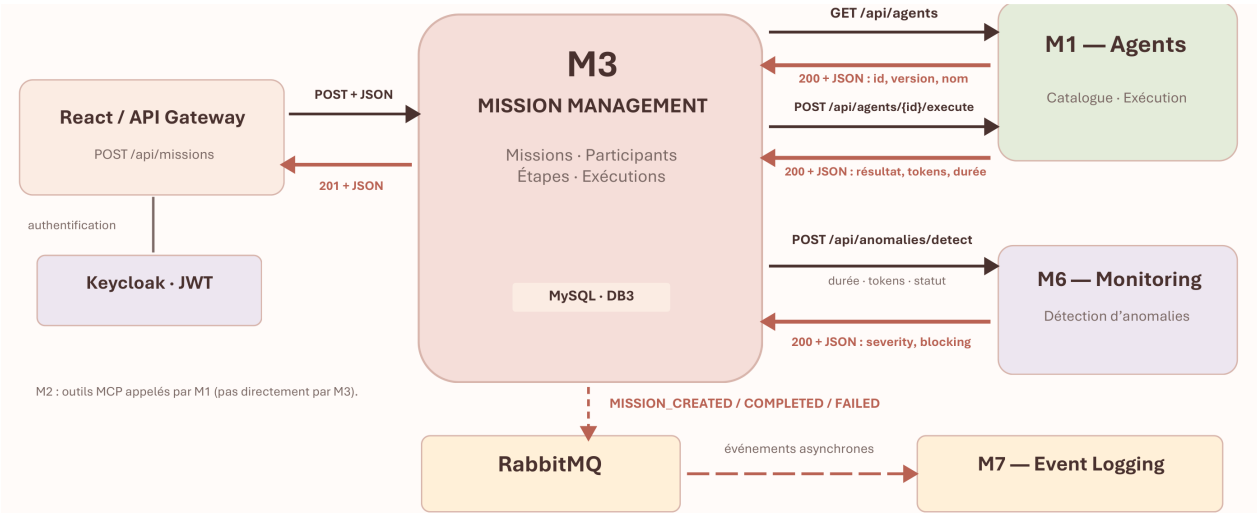


Figure 7.5 – Interactions de M3 : M1, M6 et événements RabbitMQ vers M7 (diapositive 28)

Lecture du diagramme M3 et scénario d'exécution

Une mission créée dans M3 référence des *agents existants* gérés par M1 ; M3 ne duplique ni le modèle ni la configuration métier de ces agents. Le constructeur de mission organise les étapes, les participants et les conditions de passage. Lorsqu'une exécution commence, M3 charge les étapes puis déclenche l'exécution des agents via M1. Avant publication, M3 sollicite M6 pour disposer d'un diagnostic. Les événements de début, de réussite, d'erreur ou de fin passent par RabbitMQ vers M7. Cette distinction sépare clairement *orchestration*, *exécution d'agent* et *traçabilité*.

7.4 M4 — Benchmark & Safety

7.4.1 Responsabilité et modèle

M4 est « l'examinateur » : il est le **seul** à donner les scores officiels. Il prépare des défis (une question et la réponse attendue), corrige la réponse de l'agent et attribue un score sur 100, puis vérifie la

sécurité (respect des règles, résistance aux pièges). Sa base est MySQL.
Deux types de questions existent : `OPTIMIZATION` (pour s'entraîner) et `VALIDATION` (l'examen final).

Table 7.7 – Entités principales de M4

Entité	Attributs
`Benchmark`	id, name, weightSuccess, weightLatency, weightCost
`Challenge`	id, code, title, prompt, scoringType, expectedKeywords, split, timeoutMs
`Evaluation`	id, agentVersionId, status, score, latencyMs, costEstimate, errorMessage
`SafetyTestCase`	id, type
`SafetyResult`	id, agentVersionId, verdict, observedToolCalls
Énumérations	ScoringType (KEYWORD, JSON) ; Split (OPTIMIZATION, VALIDATION) ; EvaluationStatus (PASSED, FAILED, ERROR) ; SafetyVerdict (PASS, FAIL, ERROR) ; SafetyType (FORBIDDEN_TOOL, INVALID_PARAMS, PROMPT_INJECTION, TIMEOUT, SERVICE_UNAVAILABLE, SENSITIVE_ACTION)

7.4.2 API REST

Table 7.8 – API REST de M4 (préfixe /api)

Méthode	URI	Rôle	Code
POST / GET	`/benchmarks`	Créer / lister	201 / 200
POST	`/benchmarks/{id}/challenges`	Ajouter un défi	201
POST	`/benchmarks/runs`	Lancer un run	202
GET	`/benchmarks/runs/{runId}`	Lire le statut et le score sur 100	200
GET	`/evaluations?agentVersionId=`	Voir les résultats	200
GET	`/benchmarks/{id}/compare`	Comparer les versions	200
POST	`/safety-tests`	Créer un test de sécurité	201
POST	`/safety-tests/run`	Lancer les tests de sécurité	202
GET	`/safety-results`	Voir les verdicts	200

7.4.3 Communications et pannes

M4 exécute les agents via **M1** (`POST .../execute`), est appelé par **M5** (lancement de runs et lecture des scores) et publie `BATTLE_EVALUATED` et `SAFETY_TEST_COMPLETED` vers **M7**. **Règle de sûreté** : si **M1** ne répond pas, l'évaluation passe en `ERROR`, *jamais* en `PASSED`.

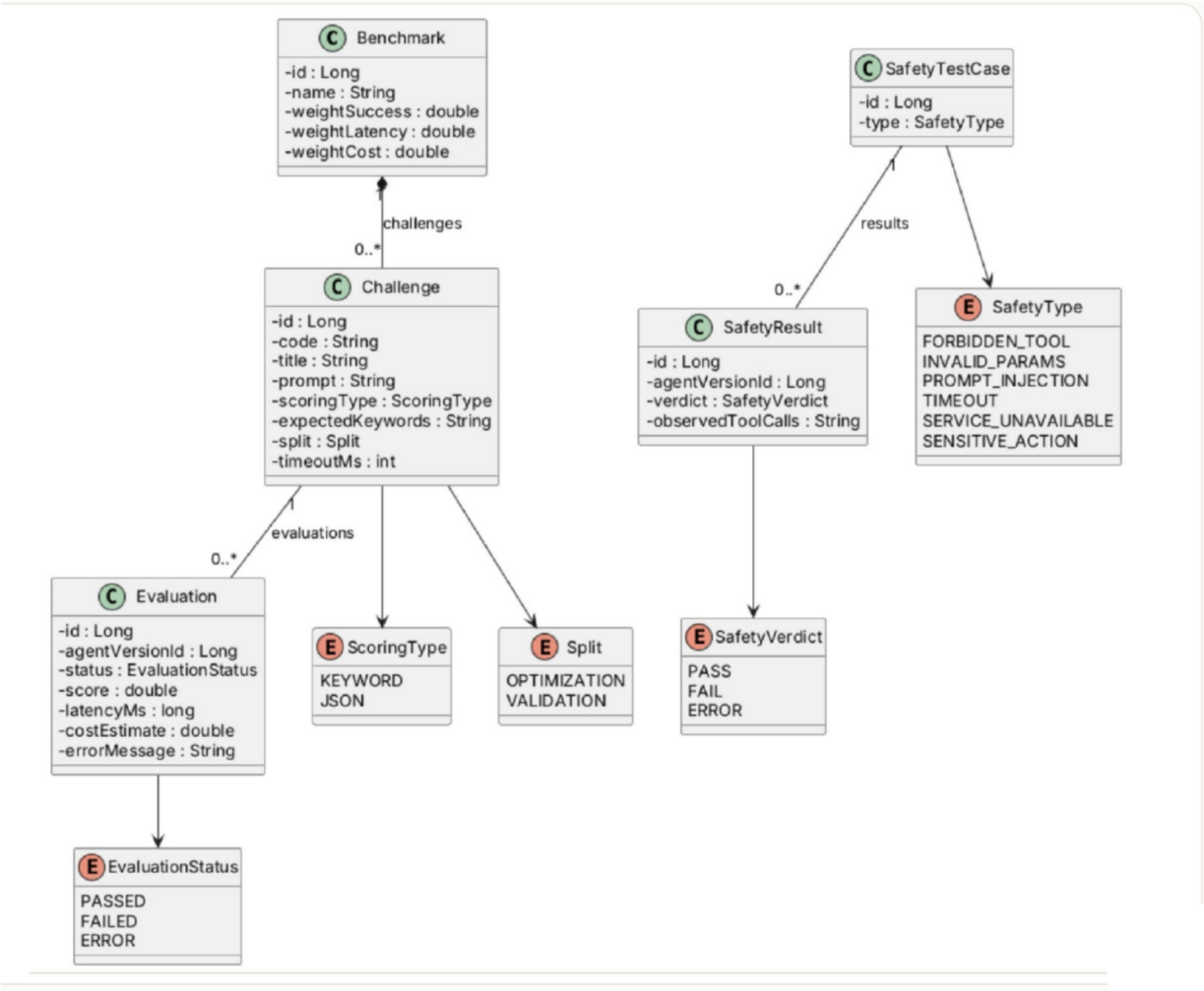


Figure 7.6 – Modèle de classes de M4 : benchmarks, jeux de test et évaluations (diapositive 31)

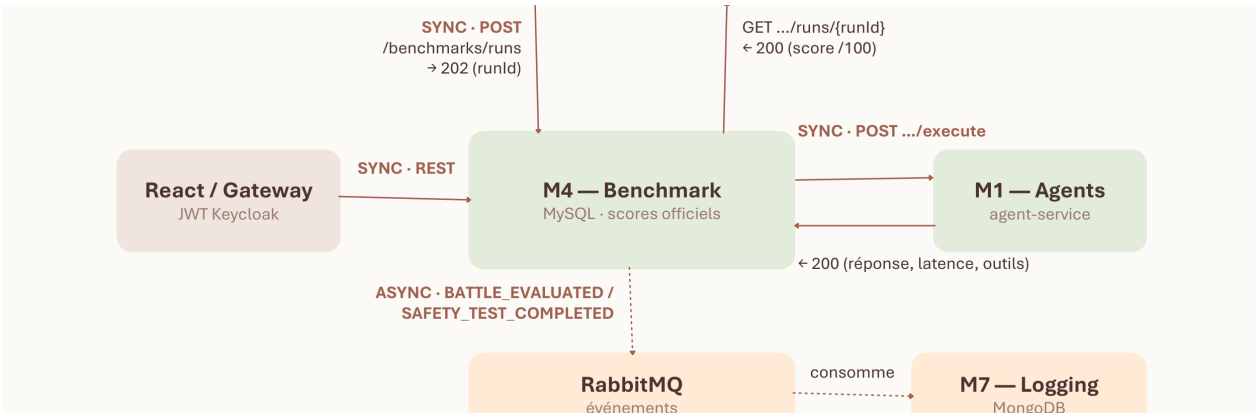


Figure 7.7 – Flux de benchmarking entre M4, M1 et M5 (diapositive 32)

7.5 M5 — Optimization (Optuna)

7.5.1 Responsabilité et modèle

M5 trouve la meilleure configuration d'un agent IA à partir des scores de **M4** en lançant des études Optuna et en proposant des configurations candidates. Il ne calcule aucun score (**M4**) et n'exécute aucun agent (**M1**) : il pilote la recherche, à la demande. Sa base est PostgreSQL (essais et scores relationnels, paramètres variables en JSONB).

Table 7.9 – Entités principales de M5

Entité	Attributs
`OptimizationStudy`	id, name, agentId, baseAgentVersionId, benchmarkId, searchSpace (JSON), maxTrials, status, createdAt, finishedAt
`Trial`	id, trialNumber, params (JSON), benchmarkRunId, score, status, startedAt, finishedAt
`BestConfiguration`	id, params (JSON), score, appliedToM1, createdAt
Énumérations	StudyStatus (PENDING, RUNNING, COMPLETED, FAILED) ; TrialStatus (RUNNING, COMPLETE, FAILED)

7.5.2 API REST et événements

Table 7.10 – API REST de M5

Méthode	URI	Description	Codes
POST	`/api/optimizations`	Lancer une étude (name, agentId, benchmarkId, searchSpace, maxTrials)	202, 400, 404, 503
GET	`/api/optimizations`	Lister les études	200
GET	`/api/optimizations/{id}`	Statut et progression	200, 404
GET	`/api/optimizations/{id}/trials`	Essais et scores	200, 404
GET	`/api/optimizations/{id}/best`	Meilleure configuration	200, 404, 409
POST	`/api/optimizations/{id}/apply`	Proposer la config à M1	201, 404, 409, 503
DELETE	`/api/optimizations/{id}`	Annuler / supprimer l'étude	204, 404

Événements publiés vers **M7** : `OPTIMIZATION_COMPLETED` (studyId, bestTrialId, score) et `OPTIMIZATION_FAILED` (studyId, reason). Le code 409 indique que l'étude n'est pas terminée.

7.5.3 Choix des modes de communication

M4 et **M1** sont appelés de façon *synchrone* (Optuna a besoin du score pour choisir l'essai suivant ; la lecture des versions et l'enregistrement de la configuration validée exigent une réponse). **M7** est alimenté de façon *asynchrone*, afin de journaliser sans jamais bloquer l'optimisation.

7.5.4 Gestion des pannes

Table 7.11 – Gestion des pannes de M5

Situation	Mécanisme	Résultat
M4 indisponible	Timeout + retry ×3 (Resilience4j, attente progressive)	Essai `FAILED`, l'étude continue ; 3 échecs consécutifs : étude `FAILED` + événement `OPTIMIZATION_FAILED`
M1 indisponible	Timeout + fallback Feign	503 ; la meilleure config reste dans **M5** (`appliedToM1 = false`) et peut être réappliquée
M7 / RabbitMQ indisponible	Accusé de réception + retry, messages persistants	Événement mis en attente puis rejoué ; l'optimisation ne dépend pas de **M7**
M5 redémarre ou tombe	Reprise au démarrage	Études `RUNNING` repassées en `FAILED` (`M5_RESTART`), relançables

L'optimisation est une tâche de fond : un service en panne ne bloque jamais les missions **M3** ni l'utilisateur.

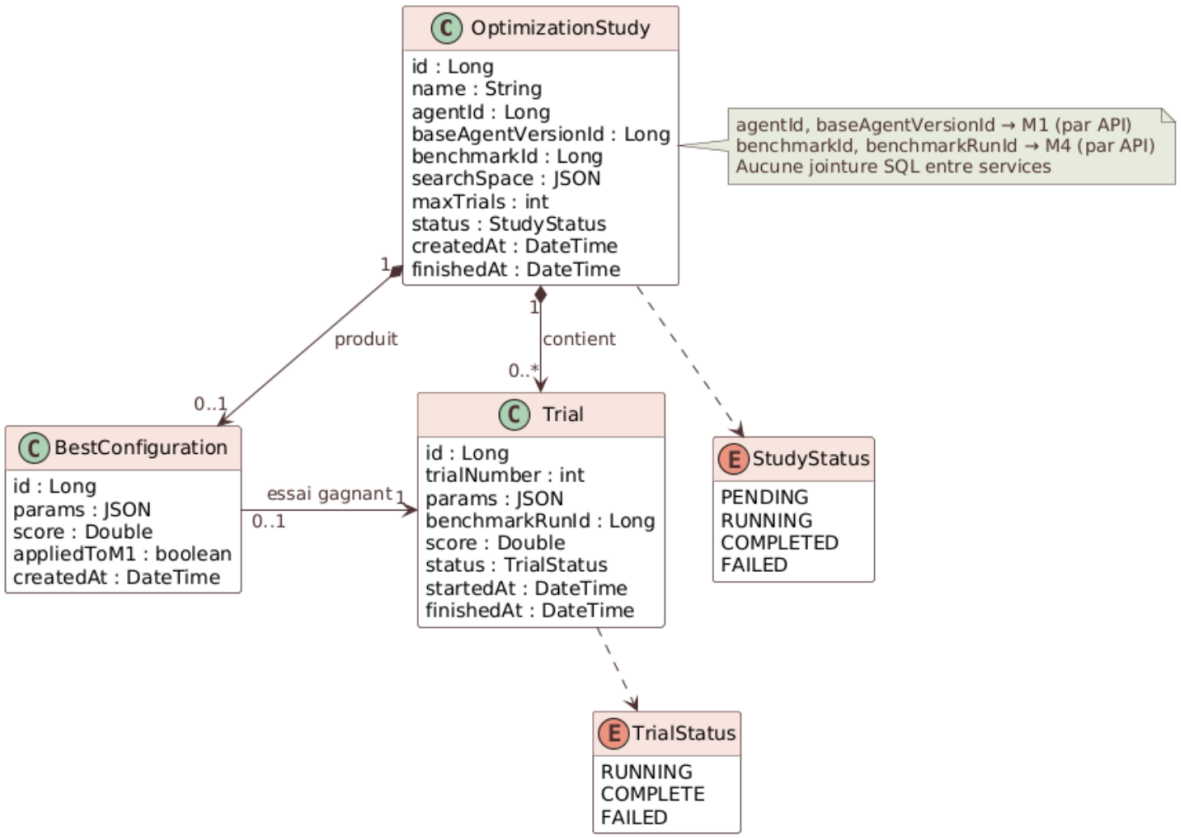


Figure 7.8 – Diagramme de classes de M5 : études, essais et configurations (diapositive 35)

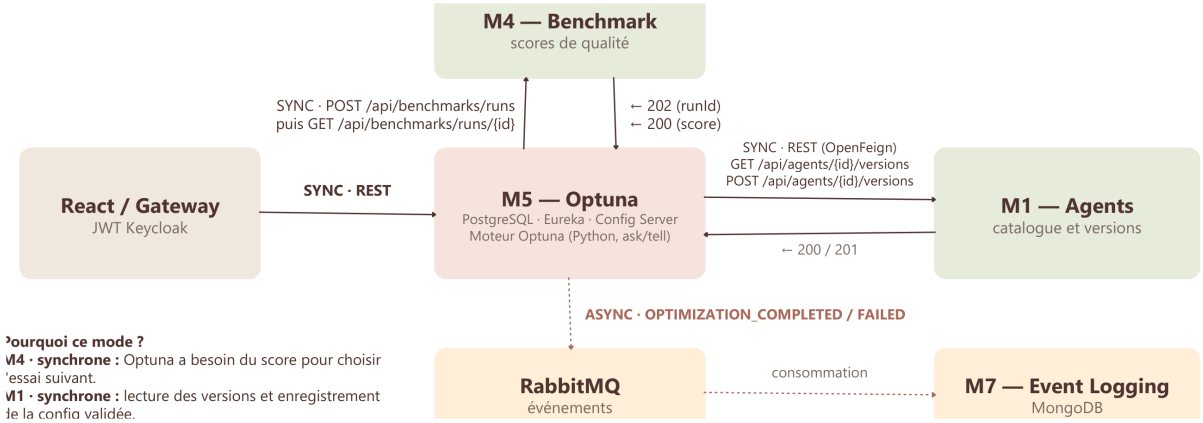


Figure 7.9 – Échanges de M5 avec M4, M1 et M7 (diapositive 34)

7.6 M6 — Monitoring & Anomaly Detection

7.6.1 Responsabilité

M6 détecte les anomalies *avant* la publication de la réponse. Le principe se déroule en trois temps :

1. **Recevoir les métriques** : **M3** envoie durée, tokens, tentatives et statut ;
2. **Calculer le score** : des seuils déterministes et un modèle *Isolation Forest* (scikit-learn) donnent un score d'anomalie ;
3. **Classer et alerter** : gravité `NORMAL`, `MINOR` ou `CRITICAL`, création d'une alerte et événement vers **M7**.

Deux sources de détection coexistent : `RULE` (erreurs bloquantes, seuils déterministes) et `ISOLATION\_FOREST`

(comportements inhabituels). Le modèle est entraîné à l'avance ; la base est H2.

7.6.2 Modèle de données

Table 7.12 – Entités principales de M6

Entité	Attributs
`AnomalyCheck`	id, executionId, agentId, agentVersionId, durationMs, tokensUsed, retryCount, executionStatus, anomalyScore, anomalyDetected, severity, reason, blocking, source, checkedAt
`Alert`	id, severity, message, status, createdAt, acknowledgedAt
`ModelVersion`	id, name, algorithm, contamination, thresholdScore, features (JSON), active, trainedAt
Énumérations	Severity (NONE, MINOR, CRITICAL) ; AnomalyReason (NONE, EXECUTION_FAILURE, HIGH_LATENCY, HIGH_TOKEN_USAGE, EXCESSIVE_RETRIES, STATISTICAL_OUTLIER) ; DetectionSource (RULE, ISOLATION_FOREST) ; AlertStatus (OPEN, ACKNOWLEDGED, RESOLVED)

7.6.3 Diagnostic et décision de gravité

Table 7.13 – Décision de gravité et action de M3

Diagnostic	Critère (exemples)	Source	Action de M3
NORMAL	Score sous le seuil, exécution `COMPLETED`	—	Mission `COMPLETED`, réponse finale publiée
MINOR	Latence ou tokens au-dessus de la normale	`ISOLATION\_FOREST`	`COMPLETED` si le Reviewer valide ; alerte consignée
CRITICAL	Échec d'exécution, tentatives maximales dépassées	`RULE`	Mission `FAILED` ; résultat candidat non diffusé

Règle essentielle. Une anomalie statistique ne prouve pas que la réponse est fausse. **M6** signale un comportement inhabituel ; le Reviewer et les règles métier jugent le résultat. Une relance d'agent reste une décision de **M3**.

7.6.4 API REST et événements

Table 7.14 – API REST de M6

Méthode	URI	Description	Codes
POST	`/api/anomalies/detect`	Analyser les métriques d'une exécution et renvoyer le diagnostic	200, 400, 503
GET	`/api/anomalies`	Lister les diagnostics (filtres : severity, executionId)	200
GET	`/api/anomalies/{id}`	Détail d'un diagnostic	200, 404
GET	`/api/alerts`	Lister les alertes (filtre : status)	200
PUT	`/api/alerts/{id}/acknowledge`	Acquitter une alerte	200, 404, 409
GET	`/api/models/active`	Version du modèle actif	200, 404

Événement publié vers **M7** : `ANOMALY_DETECTED` (executionId, severity, reason).

7.6.5 Gestion des pannes

Table 7.15 – Gestion des pannes de M6

Situation	Mécanisme	Résultat
M6 indisponible / timeout	Timeout + repli documenté	503 ; **M3** applique le repli (erreur contrôlée ou revue manuelle)
M1 indisponible	Timeout + fallback Feign	Diagnostic rendu avec les métriques reçues ; jamais de relance automatique
M7 / RabbitMQ indisponible	Accusé + retry, messages persistants	Événement rejoué ; le diagnostic ne dépend pas de **M7**
M6 redémarre / modèle absent	Rechargement du modèle actif, sinon règles seules	Erreurs bloquantes toujours détectées (`RULE`) ; H2 en mode fichier

Le diagnostic est synchrone et rapide : un service en panne ne bloque jamais **M3** indéfiniment.

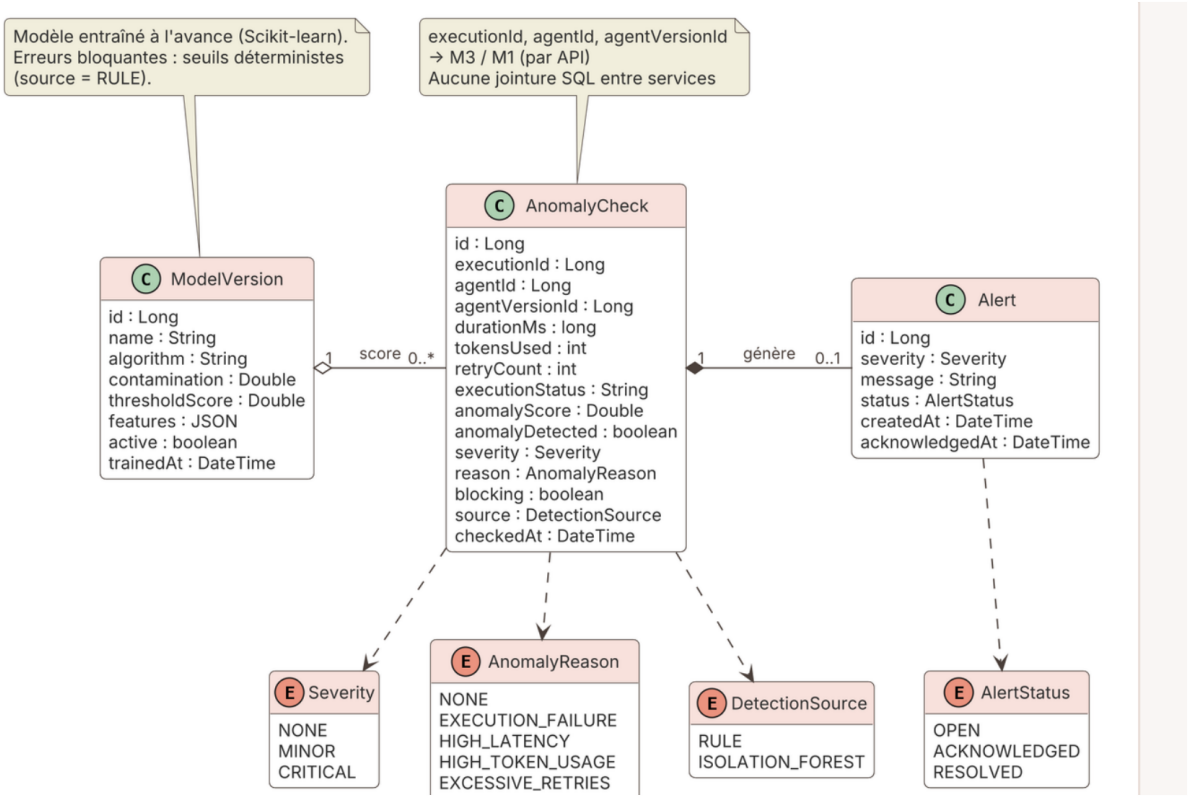


Figure 7.10 – Modèle de classes de M6 : métriques, diagnostics et alertes (diapositive 40)

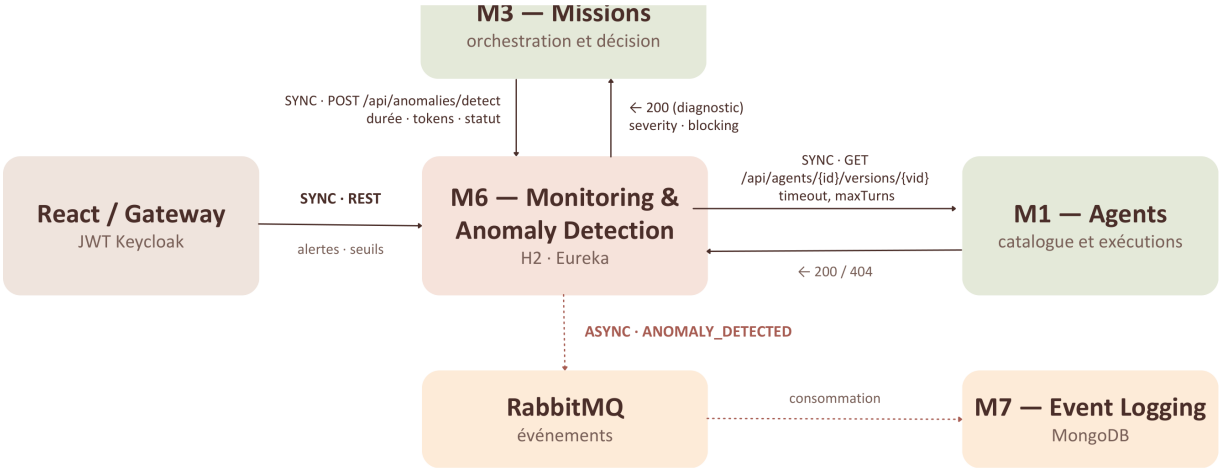


Figure 7.11 – Interactions de surveillance de M6 avec M3, M1 et M7 (diapositive 39)

7.7 M7 — Event Logging & Analytics

Statut : microservice commun inclus dans la conception cible de la plateforme ; ses fonctionnalités sont décrites ci-dessous, même si leur réalisation doit être validée par les travaux d'intégration de l'équipe.

7.7.1 Responsabilité et modèle

M7 est le service commun de traçabilité : il consomme les événements publiés par **M1** à **M6**, les historise dans MongoDB, puis les expose en consultation et en statistiques. Il **observe sans piloter** : il n'émet aucun événement métier de pilotage et n'appelle aucun autre microservice métier.

Le document `EventLog` contient : `eventId` (unique, déduplication), `eventType`, `sourceService` (M1 à M6), `correlationId` (suivi entre services), `payload` (charge utile de forme variable) et `occurredAt`.

Le DTO `AnalyticsSummary` agrège totalEvents, byService et byType.

7.7.2 Architecture interne et persistance

La chaîne de journalisation suit le parcours **producteur M1–M6 → exchange RabbitMQ → file durable → consommateur M7 → collection MongoDB**. Le producteur transmet un événement technique sans attendre que M7 ait terminé son traitement. M7 vérifie le schéma reçu, contrôle l'unicité de `eventId`, enregistre un document dans MongoDB puis confirme la consommation. Pour la consultation, ses API lisent uniquement sa propre base : aucune jointure interservices ni appel synchrone aux producteurs n'est nécessaire.

Le stockage de l'événement comprend un identifiant stable, le type, le service émetteur, un instant d'occurrence et une charge utile. Un `correlationId` permet de rapprocher les traces d'une même mission, d'un benchmark ou d'une optimisation. Les index recommandés concernent `eventId` (unique), `occurredAt`, `sourceService` et `correlationId`. Les champs de données sensibles ne doivent pas être transmis dans les traces en clair.

7.7.3 Exemple de déroulement de bout en bout

Lorsqu'une mission s'achève, M3 publie `MISSION_COMPLETED` avec `missionId`, `executionId` et `correlationId`. RabbitMQ conserve l'événement dans la file de journalisation, puis M7 l'insère en base MongoDB. L'utilisateur peut ensuite consulter l'historique des missions depuis une page dédiée, grâce aux endpoints de lecture. Si M7 est momentanément indisponible, le déroulement métier de M3 reste indépendant ; les événements sont traités à sa reprise si la configuration de persistance et de confirmation est correcte.

7.7.4 Événements consommés

Table 7.16 – Événements publiés dans RabbitMQ

Service émetteur	Événements
M1 — Agents	`AGENT_CREATED`, `AGENT_EXECUTED`, `AGENT_UPDATED`
M2 — Outils & MCP	Exécution des outils (types à préciser)
M3 — Missions	`MISSION_CREATED`, `MISSION_COMPLETED`, `MISSION_FAILED`
M4 — Benchmark	`BATTLE_EVALUATED`, `SAFETY_TEST_COMPLETED`
M5 — Optimisation	`OPTIMIZATION_COMPLETED`, `OPTIMIZATION_FAILED`
M6 — Monitoring	`ANOMALY_DETECTED`

7.7.5 API REST (lecture seule)

Table 7.17 – API REST de M7

Méthode	URI	Description	Codes
GET	`/api/events`	Lister et filtrer (service, type, dates)	200, 400
GET	`/api/events/{id}`	Détail d'un événement	200, 404
GET	`/api/analytics/summary`	Statistiques de la plateforme	200, 400

7.7.6 Gestion des pannes

Table 7.18 – Gestion des pannes de M7

Situation	Mécanisme	Résultat
RabbitMQ indisponible	Messages persistants + retry côté émetteurs	M7 reprend au retour de RabbitMQ
M7 indisponible	Messages en attente dans la queue de M7	Événements conservés si les queues/messages sont durables et les publications confirmées
Échec de sauvegarde MongoDB	Accusé après sauvegarde, puis retry	Message remis ; après plusieurs essais : file d'erreurs
Événement reçu deux fois	`eventId` unique	Un seul `EventLog` par `eventId`

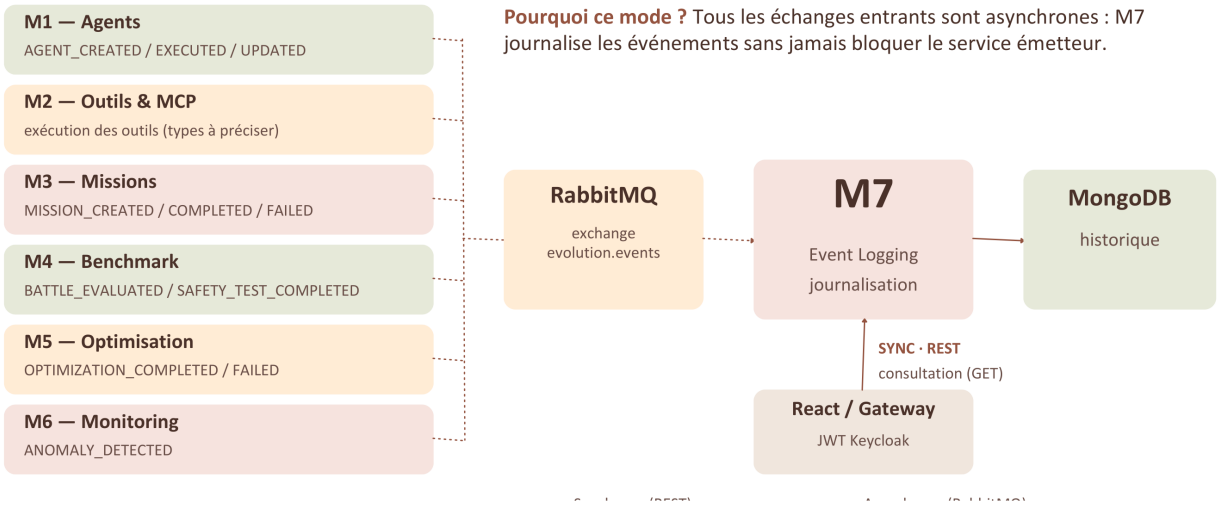


Figure 7.12 – Chaîne de traçabilité : événements métier, RabbitMQ, M7 et MongoDB (diapositive 43)

8 Sécurité

L'accès aux services est protégé de la manière suivante :

- toutes les requêtes passent par l'**API Gateway** ;
- l'authentification repose sur un jeton **JWT** émis par **Keycloak**, avec gestion des rôles (Utilisateur, Administrateur) ;
- les services sont enregistrés dans **Eureka** ;
- les codes de réponse standard s'appliquent : **401** sans jeton, **403** sans le bon rôle ;
- **M7** est en **lecture seule** : aucun endpoint d'écriture n'est exposé ;
- **M2** contrôle les outils accessibles à chaque agent (autorisations et politiques d'accès), et **M4** teste la résistance aux pièges (injection de prompt, outil interdit, action sensible).

9 Résilience et gestion des pannes

La conception suit trois principes transverses :

1. **Aucune attente infinie** : chaque appel synchrone a un timeout et, si nécessaire, des tentatives bornées (Resilience4j, fallback Feign) ;
2. **Ne jamais inventer un résultat** : en cas d'indisponibilité de **M6**, aucun diagnostic n'est fabriqué ; en cas d'indisponibilité de **M1**, une évaluation est `ERROR`, jamais `PASSED` ;
3. **Découpler l'observation de l'exécution** : la journalisation est asynchrone et persistante, afin que la panne de **M7** soit isolée des services métier ; une panne de RabbitMQ nécessite néanmoins un mécanisme de mise en attente côté producteur.

Table 9.1 – Synthèse de la résilience par dépendance

Appelant	Dépendance en panne	Comportement
M3	**M1**	Timeout, retry borné, mission en attente ou `FAILED`
M3	**M6**	503, repli documenté (erreur contrôlée ou revue manuelle)
M4	**M1**	Évaluation en `ERROR`
M5	**M4**	Retry $\times 3$, puis étude `FAILED`
M5	**M1**	503, configuration conservée et réapplicable
M1–**M6**	**M7** / RabbitMQ	Événements persistants rejoués

10 Risques et solutions

Table 10.1 – Risques prioritaires et mesures

N°	Risque	Description	Solution
01	APIs incompatibles	**M3** ne parvient pas à communiquer avec **M1**	Définir les contrats REST / JSON dès S1–S3
02	Service indisponible	**M6** ne répond pas lors de la vérification	Prévoir timeout, erreurs et statut de mission clair
03	Retard d'intégration	Un microservice est terminé trop tard pour être relié	Suivi des tâches, tests progressifs, intégration tôt

Stratégie de l'équipe : contrats clairs, tests réguliers et suivi d'équipe.

11 Technologies du projet

Table 11.1 – Stack technique de AI Agent Evolution Lab

Domaine	Technologies
Front-end	React
Back-end	Java, Spring Boot, Spring Data JPA, API REST, OpenFeign
Architecture microservices	Spring Cloud Gateway, Eureka, Config Server
Sécurité	Keycloak, JWT
IA & Agents	LLM, Python, LangGraph, MCP, Optuna, scikit-learn, MLflow
Bases de données	H2, MySQL, PostgreSQL, MongoDB
Communication & déploiement	RabbitMQ, Docker, Docker Compose
Collaboration	Git, GitHub

12 Planning du mini-projet

La feuille de route couvre les séances 4 à 10 et vise une application intégrée présentée en séance 10.

Table 12.1 – Feuille de route (séances 4 à 10)

Séance	Objectif	Technologie / jalon
S4	Validation de la conception	Jalon 1
S5	API Gateway	Routage
S6	Communication synchrone	REST
S7	Communication asynchrone	RabbitMQ
S8	Sécurité	Keycloak
S9	Conteneurisation	Docker
S10	Démo et soutenance	Jalon 2

13 Conclusion et perspectives

AI Agent Evolution Lab est conçue pour gérer le cycle de vie des agents IA : de leur configuration à leur orchestration, leur évaluation et leur surveillance. Trois apports se dégagent :

1. **Architecture modulaire** : six microservices métier et un service commun de traçabilité, chacun propriétaire de ses données ;
2. **Exécution fiable** : orchestration des missions et vérification des anomalies avant la réponse ;
3. **Amélioration continue** : benchmarks et optimisation des configurations des agents (60 % $\rightarrow$ 85 % de rapports conformes sur le test réalisé).

Perspectives. Les prochaines étapes consistent à harmoniser les contrats d'API entre responsables de services (noms d'endpoints, champs JSON, codes), à finaliser les types d'événements de **M2**, puis à réaliser l'intégration progressive (Gateway, REST, RabbitMQ, Keycloak, Docker) jusqu'à la démonstration finale.

A Exemples de messages échangés

Les exemples ci-dessous sont illustratifs ; URI, champs JSON et codes restent à valider entre les responsables de services.

A.1 Création d'une mission (Frontend/Gateway → M3)

```
POST /api/missions
{
  "name": "Analyse des ventes",
  "objective": "Rapport verifie",
  "selectionMode": "AUTOMATIC_AI",
  "maxAttempts": 3
}
-> 201 { "id": 5 }
```

A.2 Exécution d'un agent (M3 → M1)

```
POST /api/agents/12/execute
{
  "agentVersionId": 24,
  "input": "Analyse les ventes",
  "context": "mission=5; etape=2"
}
-> 200 { "status": "COMPLETED", "tokensUsed": 850, "durationMs": 2400 }
```

A.3 Contrôle technique (M3 → M6)

```
POST /api/anomalies/check
{
  "executionId": 31,
  "durationMs": 2400,
  "tokensUsed": 850,
  "status": "COMPLETED"
}
-> 200 { "severity": "LOW", "blocking": false }
```

A.4 Événement de fin de mission (M3 → RabbitMQ → M7)

```
{
  "event": "MISSION_COMPLETED",
  "source": "M3",
  "missionId": 5,
  "executionId": 31,
  "status": "COMPLETED",
  "totalTokensUsed": 1850
}
```

A.5 Lancement d'un run de benchmark (M5 → M4)

```
POST /api/benchmarks/runs
{ "benchmarkId": 7, "agentId": 12,
  "params": {"temperature":0.3,"maxTokens":1024,"style":"concise"} }
-> 202 { "runId": 88 }

GET /api/benchmarks/runs/88
-> 200 { "runId": 88, "status": "COMPLETED", "score": 85 }
```

A.6 Lancement d'une étude d'optimisation (Gateway → M5)

```
POST /api/optimizations
{ "name": "Support client v2", "agentId": 12,
  "benchmarkId": 7, "maxTrials": 20,
  "searchSpace": {
    "temperature": {"type":"float","low":0.0,"high":1.0},
    "maxTokens": {"type":"int","low":256,"high":2048},
    "style": {"type":"categorical","choices":["concise","detaille"]} } }
-> 202 { "id": 5, "status": "PENDING" }
```

A.7 Application d'une configuration (M5 → M1)

```
POST /api/agents/12/versions
{ "basedOnVersionId": 3, "source": "M5", "studyId": 5,
  "params": {"temperature":0.3,"maxTokens":1024,"style":"concise"} }
-> 201 { "versionId": 4 }
Panne M1 -> 503, config conservee (appliedToM1 = false)
```

A.8 Événements de M5 vers M7

```
{ "event": "OPTIMIZATION_COMPLETED", "source": "M5",
  "studyId": 5, "bestTrialId": 14, "score": 85,
  "timestamp": "2026-11-12T10:30:00Z" }

{ "event": "OPTIMIZATION_FAILED", "source": "M5",
  "studyId": 5, "reason": "M4_UNAVAILABLE" }
```

A.9 Diagnostic M6 → M3

```
{ "anomalyDetected": true,
  "severity": "CRITICAL",
  "reason": "EXECUTION_FAILURE",
  "blocking": true }
```

A.10 Consultation de M7

```
GET /api/events?sourceService=M5&eventType=OPTIMIZATION_FAILED
-> 200 { "content": [ ... ], "page": 0, "size": 20, "totalElements": 2 }

GET /api/analytics/summary?from=2026-11-01&to=2026-11-12
-> 200 { "totalEvents": 128,
         "byService": { "M1": 30, ... },
         "byType": { "ANOMALY_DETECTED": 3 } }
```